# dlt (data load tool) Cheatsheet

`dlt` is an open-source Python library for building robust, production-ready data pipelines without the overhead of managed connectors like Fivetran. It handles schema inference, normalisation, incremental loading, state management, and destination loading out of the box.

---

## Table of Contents



---

## Installation & Setup

```
# Install core library
pip install dlt

# Install with specific destination support
pip install "dlt[bigquery]"
pip install "dlt[snowflake]"
pip install "dlt[redshift]"
pip install "dlt[duckdb]"
pip install "dlt[postgres]"
pip install "dlt[filesystem]"   # S3, GCS, Azure Blob
```

### Initialise a New Project

```
dlt init <source_name> <destination>
# Example:
dlt init my_api duckdb
```

This scaffolds a project with:

- `<source_name>.py` — your source/resource definitions
- `.dlt/config.toml` — non-secret configuration
- `.dlt/secrets.toml` — secrets (credentials, API keys)

### Project Layout

```
my_pipeline/
├── .dlt/
│   ├── config.toml      # non-secret config
│   └── secrets.toml     # credentials (gitignored)
├── my_source.py         # source & resource definitions
├── pipeline.py          # pipeline runner
└── requirements.txt
```

---

# Core Concepts

| Concept | Description |
|---|---|
| Source | A logical group of related resources (e.g. a whole API) |
| Resource | A single data stream / table (e.g. one endpoint) |
| Pipeline | Orchestrates loading from a source to a destination |
| Schema | Auto-inferred or manually defined table structure |
| State | Persistent key-value store for incremental cursors |
| Destination | Where data is written (DuckDB, BigQuery, Snowflake, etc. |

### Hello World

```
import dlt

@dlt.resource
def my_data():
    yield {"id": 1, "name": "Alice"}
    yield {"id": 2, "name": "Bob"}

pipeline = dlt.pipeline(
    pipeline_name="my_pipeline",
    destination="duckdb",
    dataset_name="my_dataset",
)

load_info = pipeline.run(my_data())
print(load_info)
```

# Building Custom Sources & Resources

### `@dlt.resource` — Single Endpoint

```
import dlt
import requests

@dlt.resource(
    name="orders",                # table name (defaults to function name)
    write_disposition="replace",  # "replace" | "append" | "merge"
    primary_key="order_id",       # used for merge deduplication
)
def get_orders(api_url: str = dlt.config.value):
    response = requests.get(f"{api_url}/orders")
    response.raise_for_status()
    yield from response.json()["data"]
```

### `@dlt.source` — Grouping Multiple Resources

```
@dlt.source(name="my_api")
def my_api_source(
    base_url: str = dlt.config.value,
    api_key: str = dlt.secrets.value,   # pulled from secrets.toml
):
    @dlt.resource(name="users", write_disposition="replace")
    def users():
        yield from _paginate(f"{base_url}/users", api_key)

    @dlt.resource(name="events", write_disposition="append")
    def events():
        yield from _paginate(f"{base_url}/events", api_key)

    return users, events
```

### Pagination Helper Pattern

```
def _paginate(url: str, api_key: str, params: dict = None):
    """Generic cursor-based paginator."""
    headers = {"Authorization": f"Bearer {api_key}"}
    params = params or {}
    while url:
        response = requests.get(url, headers=headers, params=params)
        response.raise_for_status()
        data = response.json()
        yield from data["results"]
        # Move to next page or exit
        url = data.get("next")   # None stops the loop
        params = {}              # next URL already has params encoded
```

### Yielding in Chunks (Memory Efficiency)

```
@dlt.resource
def large_dataset():
    for chunk in fetch_data_in_chunks(chunk_size=500):
        yield chunk   # yield a list — dlt processes as a batch
```

### Transformer Resources (Dependent Resources)

Use `@dlt.transformer` to fetch child records from a parent resource without pulling all IDs into memory first.

```
@dlt.resource
def users():
    yield from get_all_users()

@dlt.transformer(data_from=users)
def user_orders(user: dict):
    """Called once per user row yielded by `users`."""
    yield from get_orders_for_user(user["id"])

# Both resources share a single pipeline run
pipeline.run([users, user_orders])
```

### Parallelism

```
@dlt.resource(parallelized=True)
def parallel_resource():
    yield from heavy_api_calls()
```

Or use `dlt.helpers.parallel_map`:

```
from dlt.sources.helpers import requests
from concurrent.futures import ThreadPoolExecutor

def fetch_page(page: int):
    return requests.get(f"/data?page={page}").json()["items"]

@dlt.resource
def all_data(total_pages: int = 10):
    with ThreadPoolExecutor(max_workers=5) as pool:
        for batch in pool.map(fetch_page, range(1, total_pages + 1)):
            yield batch
```

## Incremental Loading

Incremental loading avoids re-fetching data that has already been loaded.

### `dlt.sources.incremental` — Cursor-Based

```
import dlt
from dlt.sources.incremental import Incremental
from datetime import datetime

@dlt.resource(write_disposition="append")
def events(
    updated_at: Incremental[datetime] = dlt.sources.incremental(
        cursor_path="updated_at",           # JSON key in each record
        initial_value=datetime(2020, 1, 1), # first run start
        end_value=None,                     # optional upper bound
    )
):
    params = {"updated_after": updated_at.last_value.isoformat()}
    yield from _paginate("/events", params=params)
```

How it works:

- On first run `last_value` equals `initial_value`
- After each run, dlt persists the maximum cursor value seen
- On subsequent runs, `last_value` is the persisted maximum

### Merge Write Disposition (Upsert)

```
@dlt.resource(
    write_disposition="merge",
    primary_key="id",          # unique key for upsert
    merge_key="updated_at",    # optional secondary dedup key
)
def products():
    yield from get_all_products()
```

### `last_value_func` — Custom Cursor Logic

```
@dlt.resource(write_disposition="append")
def logs(
    cursor: Incremental[int] = dlt.sources.incremental(
        "log_id",
        initial_value=0,
        last_value_func=max,   # default; use min for reverse sequences
    )
):
    yield from get_logs_since(cursor.last_value)
```

# State Management

The pipeline state is a JSON document persisted alongside data. Use it for custom bookmarks beyond simple cursor columns.

```
import dlt

@dlt.resource
def my_resource():
    # Access the resource-level state dict
    state = dlt.current.resource_state()

    last_id = state.get("last_processed_id", 0)
    records, new_last_id = fetch_records_after(last_id)

    yield from records

    # Persist updated cursor
    state["last_processed_id"] = new_last_id
```

### Source-Level vs Resource-Level State

```
# Source-level state (shared across resources in a source)
source_state = dlt.current.source_state()

# Resource-level state (isolated per resource)
resource_state = dlt.current.resource_state()
```

### Resetting State

```
# Drop state for a pipeline (forces full reload on next run)
dlt pipeline <pipeline_name> drop-state
```

# Schema Management

### Auto-Inferred Schema

dlt infers data types from the first batch and evolves the schema automatically as new columns appear.

```
# By default, schema changes are allowed:
pipeline = dlt.pipeline(pipeline_name="p", destination="duckdb")
```

### Explicit Column Hints

```
@dlt.resource(
    columns={
        "order_id":   {"data_type": "bigint",    "nullable": False},
        "amount":     {"data_type": "decimal",   "precision": 10, "scale": 2},
        "created_at": {"data_type": "timestamp", "nullable": False},
        "metadata":   {"data_type": "json"},
    }
)
def orders():
    yield from get_orders()
```

### Schema Evolution Policies

```
pipeline = dlt.pipeline(
    pipeline_name="p",
    destination="duckdb",
    schema_contract={
        "tables":  "evolve",   # "evolve" | "freeze" | "discard_row" | "discard_value"
        "columns": "evolve",
        "data_type": "freeze",
    },
)
```

| Policy | Behaviour |
|---|---|
| evolve | Auto-add new tables / columns (default) |
| freeze | Raise an exception on unexpected schema changes |
| discard_row | Drop rows that introduce new columns |
| discard_value | Strip unknown columns but keep the row |

### Exporting & Importing Schema

```
# Export inferred schema to YAML
dlt pipeline <name> schema show > schema.yaml

# Apply a manually edited schema
dlt pipeline <name> schema update schema.yaml
```

### Nested Data / JSON Flattening

dlt auto-flattens nested dicts and lists into child tables with foreign keys:

```
# Input record
{"id": 1, "user": {"name": "Alice", "email": "alice@example.com"}, "tags": ["a", "b"]}

# Results in tables:
# my_table           → id, user__name, user__email
# my_table__tags     → value, _dlt_parent_id, _dlt_list_idx
```

To disable flattening and store as raw JSON:

```
@dlt.resource(columns={"payload": {"data_type": "json"}})
def raw_events():
    for event in get_events():
        yield {"id": event["id"], "payload": event}
```

## Transformations

### In-Resource Mapping

```
@dlt.resource
def cleaned_users():
    for user in get_raw_users():
        yield {
            "id":         user["userId"],
            "email":      user["emailAddress"].lower().strip(),
            "created_at": parse_date(user["createdOn"]),
        }
```

### `add_map` (Functional Transformation)

```
def normalise(record: dict) -> dict:
    record["email"] = record["email"].lower()
    return record

users = get_users_resource().add_map(normalise)
```

### `add_filter`

```
def is_active(record: dict) -> bool:
    return record.get("status") == "active"

active_users = get_users_resource().add_filter(is_active)
```

### Renaming / Selecting Columns at Load Time

```
@dlt.resource
def orders():
    for row in get_orders():
        yield {k: v for k, v in row.items() if k in {"id", "amount", "ts"}}
```

## Destinations

### Supported Destinations

| Destination | Install extra | Notes |
|---|---|---|
| DuckDB | dlt[duckdb] | Great for local dev & testing |
| BigQuery | dlt[bigquery] | Uses GCS staging by default |
| Snowflake | dlt[snowflake] | Uses S3/GCS/Azure staging |
| Redshift | dlt[redshift] | Uses S3 staging |
| PostgreSQL | dlt[postgres] | Direct insert |
| Athena | dlt[athena] | Iceberg-compatible |
| Filesystem | dlt[filesystem] | S3, GCS, Azure Blob, local |
| MotherDuck | dlt[duckdb] | Cloud DuckDB |
| MS SQL | dlt[mssql] | |
| Databricks | dlt[databricks] | |

### Configuring a Destination

```
# .dlt/secrets.toml
[destination.bigquery]
project_id   = "my-gcp-project"
dataset_name = "raw"

[destination.bigquery.credentials]
private_key  = "-----BEGIN RSA PRIVATE KEY-----\n..."
client_email = "sa@project.iam.gserviceaccount.com"
```

```
# Programmatic config (override secrets.toml)
import dlt

pipeline = dlt.pipeline(
    pipeline_name="bq_pipeline",
    destination=dlt.destinations.bigquery(
        credentials={
            "project_id": "my-gcp-project",
            "private_key": "...",
            "client_email": "sa@project.iam.gserviceaccount.com",
        }
    ),
    dataset_name="raw",
)
```

### Filesystem / Data Lake Destination

```
# .dlt/secrets.toml
[destination.filesystem]
bucket_url = "s3://my-bucket/raw/"

[destination.filesystem.credentials]
aws_access_key_id     = "AKIA..."
aws_secret_access_key = "..."
```

```
pipeline = dlt.pipeline(
    pipeline_name="lake_pipeline",
    destination="filesystem",
    dataset_name="raw",
)
# Files land at s3://my-bucket/raw/<dataset>/<table>/*.parquet
```

## Running & Deploying Pipelines

### Running a Pipeline

```
import dlt
from my_source import my_api_source

pipeline = dlt.pipeline(
    pipeline_name="my_pipeline",
    destination="bigquery",
    dataset_name="raw",
    full_refresh=False,   # True = drop & reload everything
)

load_info = pipeline.run(my_api_source())

# Inspect results
print(load_info)
print(pipeline.last_trace)
```

### Load Info

```
load_info = pipeline.run(source)

for package in load_info.load_packages:
    print(package.package_id, package.state)
    for job in package.jobs["completed_jobs"]:
        print(job.table_name, job.file_type, job.row_counts)
```

### Running Specific Resources Only

```
source = my_api_source()

# Select individual resources from the source
pipeline.run(source.with_resources("users", "orders"))
```

### CLI

```
# Show pipeline info
dlt pipeline <pipeline_name> info

# Show loaded tables
dlt pipeline <pipeline_name> show

# Drop all pipeline data and state
dlt pipeline <pipeline_name> drop

# Trace last run
dlt pipeline <pipeline_name> trace
```

### Deployment Helpers

```
# Generate an Airflow DAG
dlt deploy <pipeline_file>.py airflow-composer

# Generate GitHub Actions workflow
dlt deploy <pipeline_file>.py github-action --schedule "0 * * * *"

# Generate a cron-based deployment
dlt deploy <pipeline_file>.py cron --schedule "0 * * * *"
```

## Testing

### Unit Testing Resources

```
# test_my_source.py
import pytest
from my_source import get_orders

def test_get_orders_returns_records():
    records = list(get_orders())
    assert len(records) > 0
    assert "order_id" in records[0]
```

### Integration Testing with DuckDB

```
import dlt
import pytest
from my_source import my_api_source

@pytest.fixture
def test_pipeline(tmp_path):
    return dlt.pipeline(
        pipeline_name="test_pipeline",
        destination="duckdb",
        dataset_name="test",
        dev_mode=True,           # Append timestamp to dataset name
        pipelines_dir=str(tmp_path),
    )

def test_pipeline_loads_data(test_pipeline):
    load_info = test_pipeline.run(my_api_source())
    assert not load_info.has_failed_jobs

    with test_pipeline.sql_client() as client:
        with client.execute_query("SELECT COUNT(*) FROM users") as cursor:
            count = cursor.fetchone()[0]
    assert count > 0
```

### Mocking API Calls

```
from unittest.mock import patch, MagicMock

def test_orders_pagination():
    mock_response = MagicMock()
    mock_response.json.return_value = {
        "data": [{"order_id": 1}],
        "next": None,
    }
    with patch("requests.get", return_value=mock_response):
        records = list(get_orders())
    assert records == [{"order_id": 1}]
```

## Best Practices

### Secrets & Configuration

- Never hard-code credentials. Use `.dlt/secrets.toml` (gitignored) or environment variables.
- Reference values with `dlt.secrets.value` / `dlt.config.value` as default parameter values — dlt injects them automatically.

```
# dlt auto-injects matching keys from secrets.toml / env vars
@dlt.source
def my_source(api_key: str = dlt.secrets.value):
    ...
```

```
# Env var override: prefix with DLT_ (double underscore = nesting)
export DLT__MY_SOURCE__API_KEY="secret123"
```

### Write Dispositions

| Scenario | Disposition |
|---|---|
| Full reload every run | replace |
| Append-only (logs, events) | append |
| Upsert by primary key | merge |
| Slow-changing dimension | merge with merge_key |

### Incremental Loading Tips

- Always set `initial_value` to avoid a full reload on very first run.

- Use `end_value` to backfill bounded historical ranges.
- Prefer `updated_at` cursors over `id`-based cursors where possible (more reliable with late-arriving data).

### Naming Conventions

- Resource names become table names — use `snake_case`.
- Column names are normalised to `snake_case` automatically.
- Avoid reserved SQL keywords as column names.

### Error Handling

```
@dlt.resource
def safe_resource():
    try:
        yield from risky_api_call()
    except requests.HTTPError as e:
        if e.response.status_code == 429:
            # Let dlt retry via standard retry helper
            raise
        raise
```

Use `dlt.sources.helpers.requests` (a `requests` Session wrapper) for built-in retry/back-off:

```
from dlt.sources.helpers import requests

session = requests.Session()
session.headers["Authorization"] = "Bearer ..."

@dlt.resource
def safe_resource():
    response = session.get("/data", timeout=30)
    response.raise_for_status()
    yield from response.json()["items"]
```

### Chunking & Memory

- Yield lists/generators rather than accumulating all records in memory.
- For very wide tables use `max_parallel_file_size` to cap staging file sizes.
- Set `row_groups_per_file` when writing Parquet to control file granularity.

### Logging & Observability

```
import logging
logging.basicConfig(level=logging.INFO)  # shows dlt internal logs

# After a run, inspect the trace
trace = pipeline.last_trace
for step in trace.steps:
    print(step.step, step.step_info)
```

## Common Patterns

### REST API with OAuth Token Refresh

```
import dlt
import requests as req
from dlt.sources.helpers import requests

def get_token(client_id: str, client_secret: str) -> str:
    resp = req.post("https://auth.example.com/token", data={
        "grant_type":    "client_credentials",
        "client_id":     client_id,
        "client_secret": client_secret,
    })
    resp.raise_for_status()
    return resp.json()["access_token"]

@dlt.source
def oauth_api(
    client_id:     str = dlt.secrets.value,
    client_secret: str = dlt.secrets.value,
    base_url:      str = dlt.config.value,
):
    token = get_token(client_id, client_secret)
    session = requests.Session()
    session.headers["Authorization"] = f"Bearer {token}"

    @dlt.resource(write_disposition="append")
    def events():
        yield from _paginate(session, f"{base_url}/events")

    return events
```

### GraphQL Source

```
import dlt
from dlt.sources.helpers import requests

QUERY = """
query GetOrders($cursor: String) {
  orders(after: $cursor, first: 100) {
    edges { node { id amount createdAt } }
    pageInfo { endCursor hasNextPage }
  }
}
"""

@dlt.resource(write_disposition="append")
def orders(api_url: str = dlt.config.value, api_key: str = dlt.secrets.value):
    session = requests.Session()
    session.headers["X-Api-Key"] = api_key
    cursor = None
    while True:
        data = session.post(api_url, json={"query": QUERY, "variables": {"cursor": cursor}}).json()
        page = data["data"]["orders"]
        yield [edge["node"] for edge in page["edges"]]
        if not page["pageInfo"]["hasNextPage"]:
            break
        cursor = page["pageInfo"]["endCursor"]
```

### Database Source (Full Extract)

```
import dlt
import sqlalchemy as sa

@dlt.source
def postgres_source(conn_str: str = dlt.secrets.value):
    engine = sa.create_engine(conn_str)

    @dlt.resource(write_disposition="replace")
    def customers():
        with engine.connect() as conn:
            yield from (dict(row) for row in conn.execute(sa.text("SELECT * FROM customers")))

    return customers
```

### Webhook / Event Stream Source

```
import dlt

@dlt.resource(write_disposition="append")
def webhook_events(events: list):
    """
    Call this from a web framework handler,
    passing the parsed payload as `events`.
    """
    yield from events

# In your Flask/FastAPI route:
# pipeline.run(webhook_events(request.json))
```

### Multi-Tenant Pipelines

```
import dlt

TENANTS = ["tenant_a", "tenant_b", "tenant_c"]

for tenant in TENANTS:
    pipeline = dlt.pipeline(
        pipeline_name=f"pipeline_{tenant}",
        destination="bigquery",
        dataset_name=tenant,
    )
    pipeline.run(my_source(tenant_id=tenant))
```

### Backfill Historical Data

```
from datetime import date, timedelta
import dlt

@dlt.resource(write_disposition="append")
def events(start_date: date, end_date: date):
    current = start_date
    while current <= end_date:
        yield from get_events_for_date(current)
        current += timedelta(days=1)

pipeline = dlt.pipeline(pipeline_name="backfill", destination="bigquery", dataset_name="raw")
pipeline.run(events(start_date=date(2023, 1, 1), end_date=date(2023, 12, 31)))
```

# Quick Reference

## Key Decorators

| Decorator | Purpose |
|---|---|
| @dlt.source | Groups multiple resources |
| @dlt.resource | Defines a single data stream / table |
| @dlt.transformer | Dependent resource, consumes another resource |

## Key Resource Parameters

| Parameter | Values | Description |
|---|---|---|
| name | string | Table name in destination |
| write_disposition | append / replace / merge | How data is written |
| primary_key | string / list | Key(s) for merge upsert |
| merge_key | string / list | Secondary dedup key |
| columns | dict | Explicit column type hints |
| parallelized | bool | Enable concurrent execution |
| max_table_nesting | int | Limit nested JSON expansion depth |

## Useful CLI Commands

```
dlt init <source> <destination>         # scaffold project
dlt pipeline <name> info                # pipeline metadata
dlt pipeline <name> show                # list loaded tables
dlt pipeline <name> trace               # last run trace
dlt pipeline <name> drop                # drop data + state
dlt pipeline <name> drop-state          # drop state only (keep data)
dlt pipeline <name> schema show         # print schema YAML
dlt deploy <file>.py airflow-composer   # generate Airflow DAG
dlt deploy <file>.py github-action \
  --schedule "0 * * * *"               # generate GH Actions workflow
```

## Environment Variable Convention

```
DLT_<SECTION>__<KEY>=value
# Nested keys use double underscore
DLT__MY_SOURCE__API_KEY=secret
DLT__DESTINATION__BIGQUERY__PROJECT_ID=my-project
```

# Further Reading

- Official docs: https://dlthub.com/docs
- GitHub: https://github.com/dlt-hub/dlt
- Verified sources (pre-built connectors): https://github.com/dlt-hub/verified-sources
- Slack community: https://dlthub.com/community